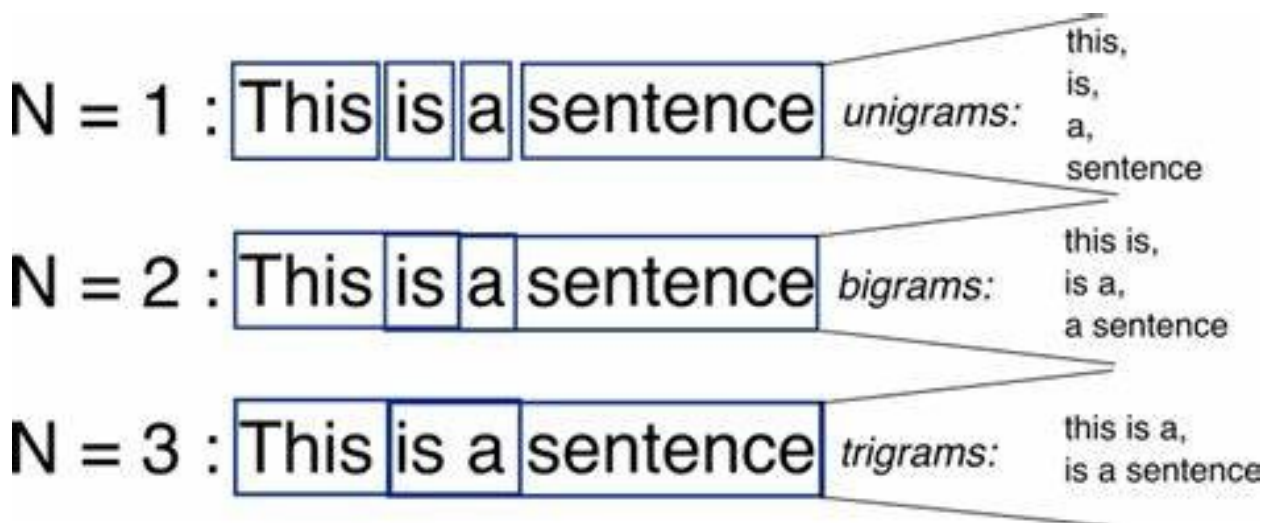


N-Grams

It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input. An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

Types of N-Grams



Use-cases of N-Grams

- Auto completion of sentences
- Auto spell check
- Voice-based personal assistant bots

Unigram

Unigram is an n-gram consisting of a single item of a sequence.

- It is commonly used to calculate the probability of finding a word in an article.

$$P(A) = \frac{\text{count of } A}{\text{count of all words}}$$

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
```

```

string = "moses supposes his toeses are roses but moses supposes
erroneously"

unigrams = ngrams(word_tokenize(string), 1)

for item in unigrams:
    print(item)

('moses',)
('supposes',)
('his',)
('toeses',)
('are',)
('roses',)
('but',)
('moses',)
('supposes',)
('erroneously',)

```

Calculating probabilities in unigram models

```

### Calculate counts of each word

string = "moses supposes his toeses are roses but moses supposes
erroneously"

unique_words = set(word_tokenize(string))

words_frq = []
for uniques in unique_words:
    frq = string.count(uniques)
    words_frq.append((uniques, frq))

words_frq

[('but', 1),
 ('roses', 1),
 ('toeses', 1),
 ('erroneously', 1),
 ('are', 1),
 ('moses', 2),
 ('supposes', 2),
 ('his', 1)]

### Calculate probabilities:

probab = []

for count in range(len(words_frq)):
    pro = words_frq[count][1]/len(word_tokenize(string))
    probab.append((words_frq[count][0],pro))

```

```
print('The probabilities are: ', probab)
```

```
The probabilities are: [('but', 0.1), ('roses', 0.1), ('toeses', 0.1), ('erroneously', 0.1), ('are', 0.1), ('moses', 0.2), ('supposes', 0.2), ('his', 0.1)]
```

Bigram

Bigram an n-gram consisting of two items of a sequence

- Two words that are used together to mean something specific

$$P(B | A) = \frac{\text{Count of } (AB)}{\text{Count of } (A)}$$

Method 1

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = nltk.bigrams(word_tokenize(string))
for i in bigrams:
    print(i)

('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Method 2

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = ngrams(word_tokenize(string), 2)

for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Trigram

Trigram an n-gram consisting of three items of a sequence

- Three words that are used together to mean something specific

Method 1

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes
erroneously"

trigrams = nltk.trigrams(word_tokenize(string))

for i in trigrams:
    print(i)

('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

Method 2

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes
erroneously"

trigrams = ngrams(word_tokenize(string), 3)
```

```

for i in trigrams:
    print(i)

('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')

```

What is padding in NLP?

Padding is a technique used to ensure that all input sequences have the same length. This is necessary because neural networks require inputs that have the same shape and size. However, when we pre-process and use texts as inputs for our model, not all sentences have the same length. In other words, some sentences are longer or shorter than others. We need to have inputs with the same size, and this is where padding comes in.

Padding is a special form of masking where the masked steps are at the start or the end of a sequence. Padding comes from the need to encode sequence data into contiguous batches: in order to make all sequences in a batch fit a given standard length. The padding is added before splitting the sentence.

n-1 padding is added.

```

from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes
erroneously"

print('----- Padding in Bigrams
-----')
list(ngrams(word_tokenize(string), pad_left=True,
left_pad_symbol("<s>",
                                pad_right=True,
right_pad_symbol("</s>", n=2))

----- Padding in Bigrams -----

[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),

```

```

('but', 'moses'),
('moses', 'supposes'),
('supposes', 'erroneously'),
('erroneously', '</s>')]

print('----- Padding in Trigrams
-----')
list(ngrams(word_tokenize(string), pad_left=True,
left_pad_symbol="<s>",
                                pad_right=True,
right_pad_symbol="</s>", n=3))

```

```

----- Padding in Trigrams -----

[('<s>', '<s>', 'moses'),
('<s>', 'moses', 'supposes'),
('moses', 'supposes', 'his'),
('supposes', 'his', 'toeses'),
('his', 'toeses', 'are'),
('toeses', 'are', 'roses'),
('are', 'roses', 'but'),
('roses', 'but', 'moses'),
('but', 'moses', 'supposes'),
('moses', 'supposes', 'erroneously'),
('supposes', 'erroneously', '</s>'),
('erroneously', '</s>', '</s>')]

```

Alternatively, We can get rid of the parameters by using

pad_both_ends method in NLTK

Note the n argument, that tells the function we need padding for bigrams.

```

from nltk.lm.preprocessing import pad_both_ends
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes
erroneously"

list(ngrams(pad_both_ends(word_tokenize(string), n=2), n=2))

[('<s>', 'moses'),
('moses', 'supposes'),
('supposes', 'his'),
('his', 'toeses'),
('toeses', 'are'),
('are', 'roses'),
('roses', 'but'),
('but', 'moses'),
('moses', 'supposes'),
]

```

```
('supposes', 'erroneously'),  
('erroneously', '</s>')]
```

Training the Maximum Likelihood Model

Maximum likelihood estimation estimates the model parameters such that the probability is maximized.

- Get the requisite `n_grams` frequency counts from a corpus.
- Normalize them to a 0-1 range.
- Build the vocabulary (of unique words): to create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words.

To do the above steps, we need to create an Ngram and add the padding then flatten the list. All these steps are requested for pre-processing to train the MLE model. However, we can use `padded_everygram_pipeline()` in NLTK to do all the steps in one call.

The `padded_everygram_pipeline()` will return:

1- **ngrams_gen**: Each sentence becomes a list of n-grams with padding.

2- **vocab_gen**: A flat stream of all tokens, with start and end symbols included

References:

<https://www.nltk.org/api/nltk.lm.html>

<https://www.kaggle.com/code/alvations/n-gram-language-model-with-nltk>

```
from nltk.lm.preprocessing import padded_everygram_pipeline  
from nltk.util import ngrams  
from nltk.tokenize import word_tokenize  
from nltk.lm import smoothing  
  
string = "moses supposes his toeses are roses but moses supposes  
erroneously"  
  
#Create Bigram Language Model  
train, vocab = padded_everygram_pipeline(2, [word_tokenize(string)])  
  
from nltk.lm import MLE  
lm = MLE(2)  
lm.fit(train, vocab) #Train the Maximum Likelihood Model
```

Calculating Probabilities

Here's how you get the score for a word given some preceding context. For example we want to know what is the chance that "b" is preceded by "a".

```
print(lm.score('supposes', ['moses'])) #Calculate Bigram Probabilities
for P('moses'|'supposes')
print(lm.score('moses', ['supposes'])) #Calculate Bigram Probabilities
for P('supposes'|'moses') - Returns 0 because unseen

1.0
0.0
```

Evaluating n_grams models using perplexity

Perplexity is a measurement of how well a probability model (like a language model) predicts a sample.

It's the probability of the test set normalized by the number of words.

A **lower** perplexity means:

- The model is less “perplexed” by the data.
- The model assigns higher probabilities to the actual words in the test set.
- Therefore, better performance.

A **higher** perplexity = more uncertainty = worse model.

```
test = [('moses', 'supposes'), ('toeses', 'are')]
print(lm.perplexity(test)) #Perplexity for bigram model

1.0
```

Task: Generating tweets using n_grams

In this Task, we will build a MLE model with n-gram to generate tweets.

The agenda is:

1. load the data: <https://www.kaggle.com/datasets/adizafar/large-random-tweets-from-pakistan>
2. pre-processing the data:
 - Remove hashtags
 - Remove RT
 - Remove any website
 - Remove any mentions
 - Remove emojis
1. build MLE model with n-gram (Bi-gram)
2. evaluate the model
3. calculate the probability of the bigram: (pakistan is)

4. calculate the perplexity of the word: (pakistan)

before we start we need to install library to handle the emojis in tweets

```
pip install emoji
```

```
pip install emoji
```

```
Requirement already satisfied: emoji in C:\Users\OmarM\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.16.0)
```

```
Note: you may need to restart the kernel to use updated packages.
```

```
import pandas as pd
import re
import emoji
import nltk
from nltk.tokenize import word_tokenize
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.lm import MLE
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\OmarM\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
True
```

```
import pandas as pd
```

```
df = pd.read_csv('Random Tweets from Pakistan.csv', encoding='utf-8',
encoding_errors='replace', low_memory=False)
```

```
tweets = df['full_text'].dropna().tolist()
```

```
df[['full_text']].head()
```

```
                                full_text
0  تیرا لیڈر میرا لیڈر نواز شریف نواز شریف کیا
1  Happy birthday to my brother n boss , May you ...
2
3  `suspicious °jikook au jimin'in yaşadığı kasab...
4  Speaking of @SpiderMan... 🤖 https://t.co/uGJ...
```

```
def clean_tweet(text):
    text = re.sub(r'#\w+', '', text) # Remove hashtags
    text = re.sub(r'\bRT\b', '', text) # Remove RT
    text = re.sub(r'http\S+|www\S+|https\S+', '', text,
flags=re.MULTILINE) # Remove websites
    text = re.sub(r'@\w+', '', text) # Remove mentions
    text = emoji.replace_emoji(text, replace='') # Remove emojis
    return text.strip().lower()
```

```
clean_tweets = [clean_tweet(tweet) for tweet in tweets]
tokenized_tweets = [word_tokenize(tweet) for tweet in clean_tweets]
```

```
print("Sample cleaned and tokenized tweet:")
print(tokenized_tweets[0])
cleaned = df['full_text'].dropna().apply(clean_tweet)
cleaned = cleaned[cleaned.str.len() > 0].drop_duplicates()
cleaned.head()
```

Sample cleaned and tokenized tweet:

```
['تیرا', 'لیڈر', 'میرا', 'لیڈر', 'نواز', 'شریف', 'نواز', 'شریف', 'کیا', 'آپکا', 'بھی', 'ہے؟؟؟']
```

```
0    تیرا لیڈر میرا لیڈر نواز شریف نواز شریف کیا...
1    happy birthday to my brother n boss , may you ...
3    `suspicious °jikook au jimin'in yaşadığı kasab...
4                                speaking of ...
5                                alexa, skip to 2021
```

Name: full_text, dtype: str

```
train_data, vocab = padded_everygram_pipeline(2, tokenized_tweets)
```

```
lm = MLE(2)
lm.fit(train_data, vocab)
print("Bi-gram MLE Model trained successfully.")
```

Bi-gram MLE Model trained successfully.

```
generated_text = lm.generate(10, text_seed=['pakistan'])
print("Generated Tweet:", " ".join(generated_text))
```

Generated Tweet: this committee , the excellent coordination and foreign affairs (

```
prob = lm.score('is', ['pakistan'])
print("Probability of 'is' given 'pakistan':", prob)
```

Probability of 'is' given 'pakistan': 0.05010239906377999

```
test_ngram = [('pakistan',)]
perplexity = lm.perplexity(test_ngram)
print("Perplexity of 'pakistan':", perplexity)
```

Perplexity of 'pakistan': 280.8649795201875